



UNIVERSITÀ DI PERUGIA
DIPARTIMENTO DI FISICA E GEOLOGIA

Analisi della Dispersione di Pacchetti d'Onda

Metodi Computazionali per la Fisica

Samuele Ricci

Obiettivi del Progetto

- ▶ **Generazione di 4 pacchetti d'onda** secondo diverse distribuzioni di frequenza e ampiezza
- ▶ **Studio dell'evoluzione temporale dei pacchetti generati** per diverse relazioni di dispersione
- ▶ **Studio dello spettro di potenza di Fourier dei pacchetti generati**

Caratteristiche del Pacchetto d'Onda

Ogni pacchetto è dato dalla sovrapposizione coerente di N componenti ondulatorie:

$$\Psi(x, t) = \sum_{i=1}^N A_i \cos(k_i x - \omega_i t)$$

- ▶ Pacchetti generati con N crescente: 10,500,1000,1500
- ▶ Frequenze estratte nel range $[1,10]$ Hz
- ▶ Relazioni di dispersione studiate: $\omega = \sqrt{ck}$, $\omega = \sqrt{ck^2}$, $\omega = \sqrt{ck^3}$, $\omega = \sqrt{b + ck^2}$

Distribuzioni di Frequenza e Ampiezza

Pacchetti generati a partire da:

- ▶ **Frequenze (ν)** estratte da 3 distribuzioni di probabilità: $p_1(\nu) = k/\nu$, $p_2(\nu) = k/\nu^2$ e una distribuzione gaussiana. Il coefficiente k è una costante di normalizzazione calcolata analiticamente
- ▶ **Ampiezze (A)** estratte da una distribuzione lineare con

$$A_{max} = \frac{a}{\sqrt{\nu}}$$

Distribuzione gaussiana simulata tramite la funzione `np.random.normal` di Numpy

Hit or Miss vs Cumulativa

Per le distribuzioni di frequenze e ampiezza non native di NumPy, sono stati valutati due approcci di campionamento:

Hit or Miss (Trial and Error)

- ▶ Generazione di coppie casuali (x, y) nell'intervallo della distribuzione.
- ▶ Accettazione solo se il punto generato giace sotto la curva.

Inversa della Cumulativa

- ▶ Trasforma variabili distribuite uniformemente in variabili distribuite secondo la funzione desiderata, senza scarto di valori.
- ▶ Richiede l'invertibilità della funzione integrale

Metodo dell'Inversa della Cumulativa

Cumulativa: Implementazione

$$c(x) = \frac{\int_a^x f(x') dx'}{\int_a^b f(x') dx'}$$

Ponendo $c(x) = y$, con $y \in [0, 1]$, il valore campionato si ottiene calcolando l'inversa $x = c^{-1}(y)$.

- **Efficienza:** A differenza dell'Hit or Miss, non presenta rigetti
- **Criticità:** Come anticipato, richiede di trovare primitive analitiche invertibili.

Listing 1: Calcolo inverse

```
if self.distr_freq == 1:
    return self.nu_min * (self.nu_max / self.nu_min)**y
elif self.distr_freq == 2:
    term_min, term_max = 1/self.nu_min, 1/self.nu_max
    return 1/(term_min - y * (term_min - term_max))
...
# Generazione ampiezze
A_max=a/np.sqrt(self.nu)
y = np.random.random(self.N)
return A_max* np.sqrt(y)
...
```

Struttura della Classe

- ▶ In ingresso N , frequenze (ν), distribuzione scelta
- ▶ **Metodi:** Gestiscono il calcolo di k e l'estrazione statistica di ν ed A
- ▶ **Output:** Restituisce un oggetto contenente le informazioni di un pacchetto di N onde, pronto per la sovrapposizione

Numeri d'onda: `k_v_onda`

Funzione esterna che calcola l'array k in base alla relazione di dispersione scelta

Listing 2: calcolo numeri d'onda

```
def k_v_onda(w, rel_disp, b, c):  
    if (rel_disp == 1):  
        return np.power(w, 2)/c  
    ...
```

Evoluzione Temporale: La Funzione `evolve_wp`

La funzione gestisce la propagazione calcolando l'ampiezza risultante $\Psi(x, t)$, restituendo un array di valori di ampiezza al variare dello spazio e del tempo

- **Input:** Riceve un'istanza di `WavePacket`, le coordinate (x, t) e i parametri della relazione di dispersione.
- **Ciclo for:** Implementa la sovrapposizione delle N onde contenute in `WavePacket`

Listing 3: Funzione di evoluzione

```
def evolve_wp(packet, x, t, rel_disp, b, c):  
    omega = w(packet.nu)  
    k_reldisp = k_v_onda(omega, rel_disp, b, c)  
  
    psi = np.zeros(np.size(x))  
    for i in range(packet.N):  
        # Somma di N onde piane  
        psi += packet.A[i] * np.cos(k_reldisp[i] * x -  
                                     omega[i] * t)  
    return psi
```

La funzione `evolve_wp` permette di visualizzare sia lo stato a $t = 0$ che l'evoluzione per più istanti di tempo.

Analisi Spettrale: Campionamento e Nyquist

Il segnale è stato campionato in un punto fisso ($x = 0$) osservando l'evoluzione temporale per 100 secondi.

Criterio di Nyquist

- **Campionamento:** 10.000 campionamenti su 100s $\rightarrow f_s = 100$ Hz. ($dt = 0.01$ s)
- Affinché un segnale con una certa ν_{max} possa essere ricostruito dai suoi campioni è necessario che $f_s > 2\nu_{max}$.
- Nel nostro caso $\nu_{max} = 10$ Hz e $f_s = 100$ Hz > 20 Hz

Listing 4: Calcolo di $\Psi(0, t)$

```
t_fft=np.linspace(0, 100, 10000)
dt = t_fft[1] - t_fft[0]
for p in packets:
    psi_start=evolve_wp(p,0,t_fft,rel_disp,b,c)
...
```

Analisi di Fourier

L'analisi spettrale permette di scomporre il segnale $\Psi(0, t)$ nelle sue componenti armoniche, verificando la corrispondenza con le distribuzioni di frequenza utilizzate.

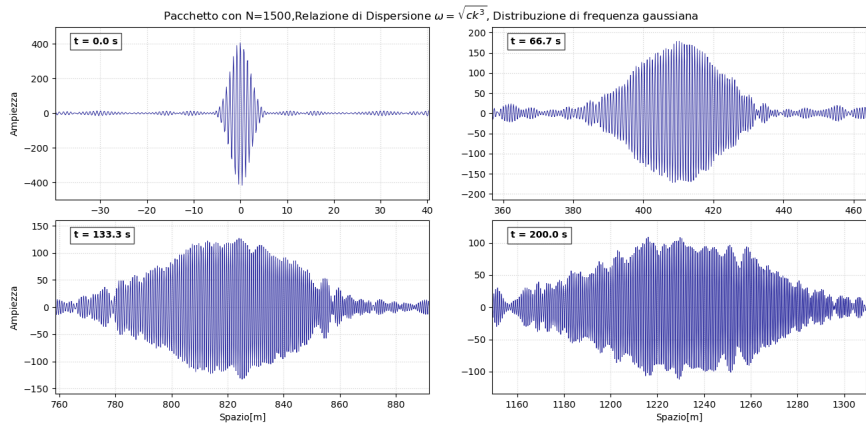
Implementazione nel codice

- **Uso di rfft:** Funzione di Scipy ottimizzata per segnali reali, evita il calcolo della parte simmetrica dello spettro. Questo porta ad un dimezzamento dei punti da calcolare, cioè maggiore efficienza e minor occupazione di memoria.
- **Spettro di potenza:** La potenza è calcolata come modulo quadro dei coefficienti $|c_k|^2$.
- **Estrazione delle frequenze:** L'asse delle frequenze è mappato tramite la funzione `rfftfreq`.

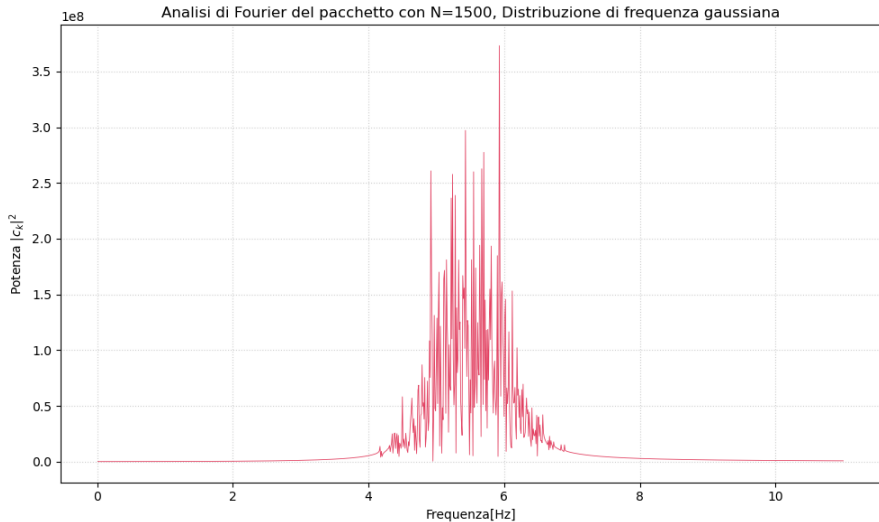
Listing 5: Calcolo delle componenti spettrali

```
c_k = fft.rfft(psi_start)
pow=np.power(np.abs(c_k),2)
freq=fft.rfftfreq(len(psi_start), d=dt)
...
mask = (freq > 0) & (freq <= v_max * 1.1)
plt.plot(freq[mask], pow[mask], color='crimson')
...
```

Evoluzione Temporale $\Psi(x, t)$



Analisi Spettrale $|c_k|^2$



Grazie per l'attenzione

Samuele Ricci